Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology

Department of

Electrical & Computer Engineering

Lab Report 2

Implementation and Evaluation of the K-Nearest Neighbors (KNN) Algorithm for Multi-Class Classification

Course Code: ECE 4246

Course Title: Machine Learning Sessional

Submitted to:
Md. Faysal Ahamed
Assistant Professor
Dept of ECE, RUET

Submitted by:
Md. Tajim An Noor
Roll: 2010025
Semester: 4th Year Even

Submission Date: February 9, 2026

Contents

1 Theory and Introduction **1**
1.1 Classification Mechanism 1
1.2 Distance Metrics 1

2 Methodology **2**
2.1 Dataset 2
2.2 Algorithm Implementation 2
2.3 Code 2

3 Results and Discussion **5**
3.1 Quantitative Results 5
3.2 Visual Analysis 6

4 Conclusion **6**

References **7**

Implementation of Linear and Logistic Regression for Regression and Classification Problems

1 Theory and Introduction

The K-Nearest Neighbors (KNN) algorithm is a non-parametric, lazy learning algorithm used for both classification and regression. Unlike eager learning methods (e.g., Logistic Regression), KNN does not build a model during the training phase. Instead, it stores the entire training dataset and performs computation only during the prediction phase [1].

1.1 Classification Mechanism

Given a new data point x_{query}, the algorithm identifies the k training samples closest to it. The class label is determined by a majority vote among its k nearest neighbors:

$$\hat{y} = \text{mode}(\{y_i : (x_i, y_i) \in N_k(x_{query})\}) \tag{1}$$

where $N_k(x_{query})$ is the set of k nearest neighbors.

1.2 Distance Metrics

The "closeness" between points is defined by a distance metric. Two common metrics were implemented in this study:

Euclidean Distance

The straight-line distance between two points p and q in an n-dimensional space:

$$d_{Euclidean}(p, q) = \sqrt{\sum_{i=1}^{n} (p_i - q_i)^2} \tag{2}$$

Manhattan Distance

The sum of absolute differences between the coordinates of the points (also known as L_1 norm):

$$d_{Manhattan}(p, q) = \sum_{i=1}^{n} |p_i - q_i| \tag{3}$$

2 Methodology

2.1 Dataset

The *Iris* dataset was used, consisting of 150 samples across three classes: *Setosa*, *Versicolor*, and *Virginica*. For visualization purposes, two features (Petal Length and Petal Width) were selected. The data was split into training (80%) and testing (20%) sets [2].

2.2 Algorithm Implementation

The algorithm was implemented using Python (NumPy) without relying on high-level libraries like Scikit-Learn.

Algorithm 1 K-Nearest Neighbors (Prediction Phase)

Input: Training Data (X_{train}, y_{train}), Query Point x_q, Hyperparameter k
Output: Predicted Class $\hat{y}$
function PREDICT(x_q)
    $distances \leftarrow []$
    for each x_i in X_{train} **do**
        $d \leftarrow \text{DistanceMetric}(x_q, x_i)$ ▷ Euclidean or Manhattan
        Append d to $distances$
    end for
    $indices \leftarrow \text{Argsort}(distances)[:k]$ ▷ Get indices of k smallest distances
    $neighbors \leftarrow y_{train}[indices]$
    $\hat{y} \leftarrow \text{Mode}(neighbors)$ ▷ Majority Vote
    return $\hat{y}$
end function

2.3 Code

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt


# 1. DISTANCE METRICS
def euclidean_distance(x1, x2):
    return np.sqrt(np.sum((x1 - x2) ** 2))


def manhattan_distance(x1, x2):
    return np.sum(np.abs(x1 - x2))


# 2. DATA LOADING & PREPROCESSING
def load_iris_3class():
    url = "https://raw.githubusercontent.com/m
    ↪ waskom/seaborn-data/master/iris.csv"
    try:
        df = pd.read_csv(url)
    except Exception as e:
        print("Failed to load data.")
        return None, None
    # 0: Setosa, 1: Versicolor, 2: Virginica
    unique_species = df["species"].unique()
    mapper = {name: i for i, name in
    ↪ enumerate(unique_species)}
    df["species"] = df["species"].map(mapper)
    # Normalize features to [0, 1]
    feature_cols = ["sepal_length",
    ↪ "sepal_width", "petal_length",
    ↪ "petal_width"]
    df[feature_cols] = (df[feature_cols] -
    ↪ df[feature_cols].min()) / (
        df[feature_cols].max() -
        ↪ df[feature_cols].min()
    )
    X = df[["sepal_length",
    ↪ "sepal_width"]].values
    # X = df[['sepal_length',
    ↪ 'petal_width']].values
    y = df["species"].values

    return X, y, list(unique_species)


def train_test_split_custom(X, y,
↪ test_size=0.2, seed=9):
```

```python
    np.random.seed(seed)
    indices = np.random.permutation(len(X))
    split = int(len(X) * test_size)
    test_idx, train_idx = indices[:split], indices[split:]
    return X[train_idx], X[test_idx], y[train_idx], y[test_idx]


# 3. KNN ALGORITHM (Raw Implementation)


class KNN:
    def __init__(self, k, metric):
        self.k = k
        self.metric = metric
        self.dist_func = (
            euclidean_distance if metric == "euclidean" else manhattan_distance
        )
        self.X_train = None
        self.y_train = None

    def fit(self, X, y):
        # KNN is "lazy": just store the training data
        self.X_train = X
        self.y_train = y

    def _predict_sample(self, x):
        # 1. Calculate distances to all training points
        distances = [self.dist_func(x, x_train) for x_train in self.X_train]
        # 2. Sort and get indices of K nearest neighbors
        k_indices = np.argsort(distances)[:self.k]
        # 3. Get labels of neighbors
        k_nearest_labels = [self.y_train[i] for i in k_indices]
        # 4. Vote (Mode) return most common label
        most_common = np.bincount(k_nearest_labels).argmax()
        # 5. Probability (for ROC): Fraction of neighbors matching prediction
        prob = np.bincount(k_nearest_labels, minlength=3) / self.k

        return most_common, prob

    def predict(self, X):
        predictions = [self._predict_sample(x) for x in X]
        # Unpack predictions (classes) and probabilities
        y_pred = np.array([p[0] for p in predictions])
        y_probs = np.array([p[1] for p in predictions])
        return y_pred, y_probs


# 4. EVALUATION METRICS (Multi-Class)
def confusion_matrix_3class(y_true, y_pred):
    cm = np.zeros((3, 3), dtype=int)
    for t, p in zip(y_true, y_pred):
        cm[t, p] += 1
    return cm


def calculate_metrics(y_true, y_pred):
    cm = confusion_matrix_3class(y_true, y_pred)
    # Accuracy: Sum of Diagonal / Total
    accuracy = np.trace(cm) / np.sum(cm)
    # Per-Class Precision, Recall, F1
    precisions = []
    recalls = []

    for i in range(3):
        tp = cm[i, i]
        fp = np.sum(cm[:, i]) - tp
        fn = np.sum(cm[i, :]) - tp

        p = tp / (tp + fp) if (tp + fp) > 0 else 0
        r = tp / (tp + fn) if (tp + fn) > 0 else 0
        precisions.append(p)
        recalls.append(r)

    # Macro Average (Simple average of per-class scores)
    macro_precision = np.mean(precisions)
    macro_recall = np.mean(recalls)
    macro_f1 = (
        2 * (macro_precision * macro_recall) / (macro_precision + macro_recall)
        if (macro_precision + macro_recall) > 0
        else 0
    )

    return accuracy, macro_precision, macro_recall, macro_f1, cm


def get_roc_curve_ovr(y_test, y_probs, class_id):
    # One-vs-Rest ROC for a specific class
    y_binary = (y_test == class_id).astype(int)
    probs_binary = y_probs[:, class_id]

    thresholds = np.linspace(0, 1.1, 20)  # 0 to 1
    tpr_list, fpr_list = [], []

    for t in thresholds:
```

```python
        pred_bin = (probs_binary >= t).astype(int)

        TP = np.sum((y_binary == 1) & (pred_bin == 1))
        TN = np.sum((y_binary == 0) & (pred_bin == 0))
        FP = np.sum((y_binary == 0) & (pred_bin == 1))
        FN = np.sum((y_binary == 1) & (pred_bin == 0))

        tpr = TP / (TP + FN) if (TP + FN) > 0 else 0
        fpr = FP / (FP + TN) if (FP + TN) > 0 else 0

        tpr_list.append(tpr)
        fpr_list.append(fpr)

    return fpr_list, tpr_list


# 5. VISUALIZATION FUNCTIONS
def plot_decision_boundary(model, X, y, title="KNN Decision Boundary"):
    # Create mesh
    x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
    y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.05), np.arange(y_min, y_max, 0.05))

    # Predict for whole mesh
    mesh_points = np.c_[xx.ravel(), yy.ravel()]
    Z, _ = model.predict(mesh_points)
    Z = Z.reshape(xx.shape)

    plt.contourf(xx, yy, Z, alpha=0.3, cmap="viridis")
    # Scatter points
    colors = ["purple", "teal", "yellow"]
    labels = ["Setosa", "Versicolor", "Virginica"]
    for i in range(3):
        idx = np.where(y == i)
        plt.scatter(X[idx, 0], X[idx, 1], c=colors[i], edgecolors="k", label=labels[i])

    plt.title(title)
    plt.xlabel("Length")
    plt.ylabel("Width")
    plt.legend()


# 6. MAIN EXECUTION
# A. Load
X, y, class_names = load_iris_3class()
# B. Split
X_train, X_test, y_train, y_test = train_test_split_custom(X, y, test_size=0.2)
print(f"Train size: {len(X_train)}, Test size: {len(X_test)}")
# C. Train Model
k_neighbors = 5
# metric = "euclidean"
metric = "manhattan"
knn = KNN(k=k_neighbors, metric=metric)
knn.fit(X_train, y_train)
print(f"Training KNN (k={k_neighbors}, metric={metric})...")
# D. Predict
y_pred, y_probs = knn.predict(X_test)
# E. Evaluate
acc, prec, rec, f1, cm = calculate_metrics(y_test, y_pred)

print("\nEVALUATION REPORT (3-Class Iris)")
print("=" * 40)
print(f"Accuracy:  {acc:.2%}")
print(f"Precision (Macro): {prec:.2%}")
print(f"Recall (Macro):    {rec:.2%}")
print(f"F1 Score (Macro):  {f1:.4f}")
print("-" * 20)
print("Confusion Matrix:")
print(cm)

# F. Plotting
plt.figure(figsize=(18, 5))

# Plot 1: Confusion Matrix
plt.subplot(1, 3, 1)
plt.imshow(cm, cmap="Blues", alpha=0.7)
for i in range(3):
    for j in range(3):
        plt.text(j, i, cm[i, j], ha="center", va="center", fontsize=14)
plt.title("Confusion Matrix")
plt.xticks(range(3), class_names)
plt.yticks(range(3), class_names)
plt.xlabel("Predicted")
plt.ylabel("Actual")

# Plot 2: ROC Curves (One-vs-Rest)
plt.subplot(1, 3, 2)
colors = ["purple", "teal", "orange"]
for i in range(3):
    fpr, tpr = get_roc_curve_ovr(y_test, y_probs, i)
    # Sort for plotting line correctly
    sorted_indices = np.argsort(fpr)
    fpr = np.array(fpr)[sorted_indices]
    tpr = np.array(tpr)[sorted_indices]

    plt.plot(fpr, tpr, color=colors[i], lw=2, label=f"{class_names[i]} vs Rest")

```

```python
plt.plot([0, 1], [0, 1], "k--", lw=1)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Multi-Class ROC (One-vs-Rest)")
plt.legend()
plt.grid(True, alpha=0.3)

# Plot 3: Decision Boundary
plt.subplot(1, 3, 3)
plot_decision_boundary(
    knn, X_test, y_test, title=f"KNN Decision Boundary (k={k_neighbors})"
)

plt.tight_layout()
# plt.savefig("./images/output/knn_euclidean_results.png")
plt.savefig("./images/output/knn_manhattan_results.png")
plt.show()
```

3 Results and Discussion

The KNN model was trained with $k = 5$ and evaluated using both Euclidean and Manhattan distance metrics on a test set of 30 samples.

3.1 Quantitative Results

The model was tested using two different distance metrics to observe the effect on classification accuracy. The console outputs below summarize the evaluation metrics for each experiment:

Euclidean:

```
Train size: 120, Test size: 30
Training KNN (k=5, metric=euclidean)...
EVALUATION REPORT (3-Class Iris)
========================================
Accuracy:  90.00%
Precision (Macro): 91.67%
Recall (Macro):    87.50%
F1 Score (Macro):  0.8953
--------------------
Confusion Matrix:
[[13  0  0]
 [ 0  9  0]
 [ 0  3  5]]
```

Manhattan:

```
Train size: 120, Test size: 30
Training KNN (k=5, metric=manhattan)...
EVALUATION REPORT (3-Class Iris)
========================================
Accuracy:  83.33%
Precision (Macro): 80.48%
Recall (Macro):    80.09%
F1 Score (Macro):  0.8028
--------------------
Confusion Matrix:
[[13  0  0]
 [ 0  7  2]
 [ 0  3  5]]
```

Performance Analysis

The Euclidean distance metric achieved a higher accuracy of 90.00% compared to the 83.33% accuracy observed with the Manhattan distance. The Confusion Matrices indicate that *Setosa* was perfectly separated from the other classes (13 correct predictions) by both metrics.

However, the separation between *Versicolor* and *Virginica* proved more challenging:

- **Euclidean Metric:** Successfully classified all *Versicolor* samples but misclassified 3 *Virginica* samples as *Versicolor*.
- **Manhattan Metric:** Struggled further, misclassifying 2 *Versicolor* samples as *Virginica* in addition to the 3 *Virginica* errors seen in the Euclidean test.

This suggests that for this dataset, the Euclidean (L_2) geometry better captures the natural clustering of the flower species than the Manhattan (L_1) geometry.

3.2 Visual Analysis

Confusion Matrix and ROC Curves

Figure 1 visualizes the performance of the best-performing model (Euclidean). The One-vs-Rest ROC curves demonstrate excellent classifier performance, particularly for *Setosa* which has an Area Under the Curve (AUC) of 1.0. *Versicolor* and *Virginica* show high AUC scores despite the slight overlap. The decision boundary visualization confirms the non-linear nature of KNN; the algorithm creates complex, irregular boundaries that adapt to the local density of the data points, which contrasts with the straight lines produced by linear classifiers.

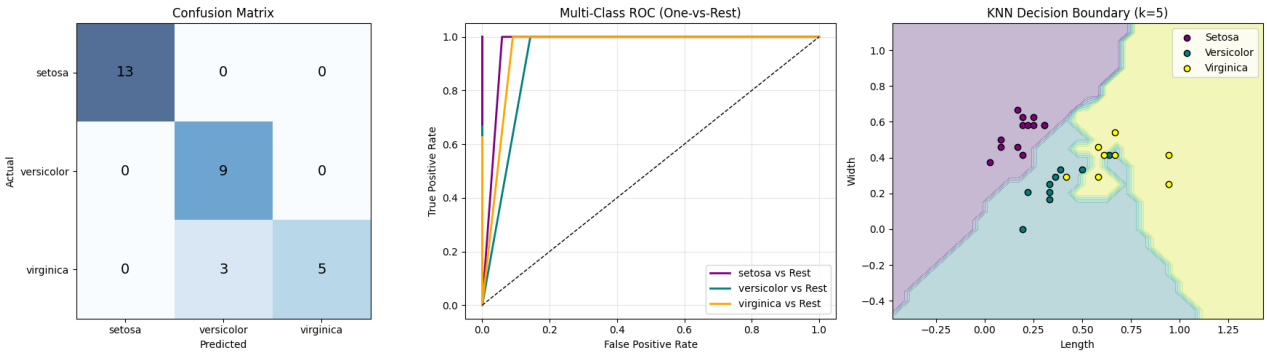

Figure 1: Performance Visualization (Euclidean Metric). Left: Confusion Matrix. Middle: Multi-Class ROC Curves (One-vs-Rest). Right: Decision Boundary visualization illustrating KNN's non-linear classification.

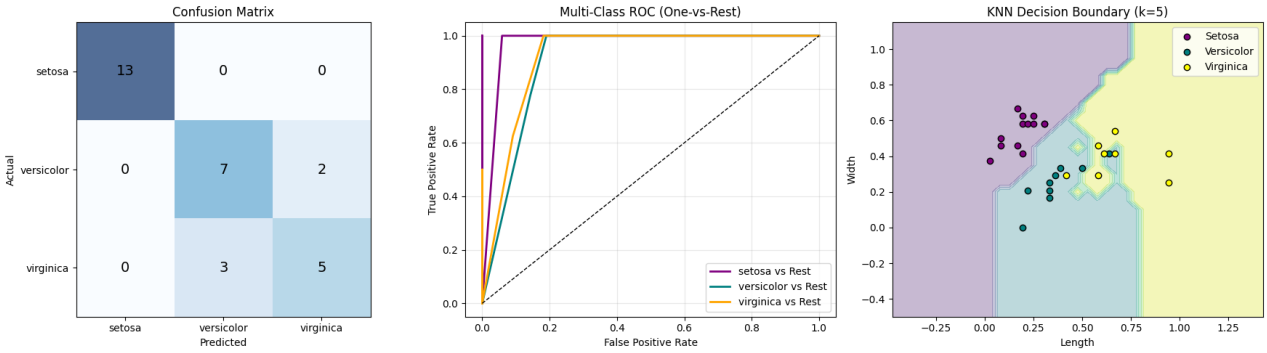

Figure 2: Performance Visualization (Manhattan Metric). Left: Confusion Matrix. Middle: Multi-Class ROC Curves (One-vs-Rest). Right: Decision Boundary visualization illustrating KNN's non-linear classification.

4 Conclusion

The implementation of the K-Nearest Neighbors algorithm successfully classified the Iris dataset. Two distance metrics were implemented and compared: Euclidean and Manhattan. The experiment highlighted that KNN is highly effective for multi-class classification, though its performance depends heavily on the chosen distance metric. In this experiment, Euclidean distance achieved 90.00% accuracy, outperforming Manhattan distance at 83.33% accuracy.

The algorithm perfectly distinguishes linearly separable classes such as Setosa but encounters minor difficulties with overlapping clusters like Versicolor and Virginica. As a lazy learning algorithm, KNN incurs zero training cost but higher prediction cost ($O(N)$), making it suitable for smaller datasets where decision boundaries are irregular and non-linear.

References

[1] C. M. Bishop, *Pattern recognition and machine learning.* springer, 2006.

[2] R. A. Fisher, "The use of multiple measurements in taxonomic problems," *Annals of eugenics*, vol. 7, no. 2, pp. 179–188, 1936.